



Politechnika
Wrocławska

Politechnika Wrocławska

Wydział informatyki i telekomunikacji

Kierunek: Informatyczne systemy automatyki (ISA)

Specjalność: Zastosowania technologii informacyjnych (IZT)

Sprawozdanie

z przedmiotu *Platformy programistyczne*

Realizacja projektu:

**Workout Tracker – aplikacja webowa do śledzenia
treningów i diety**

Autor:

Hleb Lizhbanau

Nr albumu: 282648

Prowadzący:

dr inż. Aneta Górniak

Data wykonania:

12 czerwca 2026

Spis treści

1	Wstęp	2
2	Cel i zakres projektu	2
2.1	Cel biznesowy	2
2.2	Zakres funkcjonalny (MVP)	2
3	Wymagania projektowe i ich realizacja	2
4	Architektura systemu	3
4.1	Warstwy aplikacji	3
4.2	Stos technologiczny	3
4.3	Przepływ danych przy wyszukiwaniu produktów	4
5	Model danych	4
6	Opis funkcjonalności	4
6.1	Moduł treningów (Workouts)	4
6.2	Moduł posiłków (Meals)	5
6.3	Integracja z FatSecret	5
6.4	Panel administratora (Products)	5
6.5	Rejestracja i logowanie	5
7	Konteneryzacja i środowiska uruchomieniowe	6
7.1	Docker (środowisko deweloperskie)	6
7.2	Azure (środowisko produkcyjne)	6
8	Testy	6
9	Dokumentacja techniczna	7
10	Zrzuty ekranu aplikacji	7
11	Wnioski	10

1. Wstęp

Celem projektu było zaprojektowanie i implementacja aplikacji webowej umożliwiającej użytkownikowi rejestrowanie treningów, ćwiczeń oraz posiłków wraz z analizą wartości odżywczych. Aplikacja *Workout Tracker* została zrealizowana w technologii ASP.NET Core MVC z wykorzystaniem Entity Framework Core, bazy danych Microsoft SQL Server oraz integracji z zewnętrznym API FatSecret.

Projekt obejmuje pełny cykl wytwarzania oprogramowania: model danych, warstwę logiki biznesowej, interfejs użytkownika, uwierzytelnianie z rolami, testy jednostkowe, konteneryzację Docker, dokumentację DocFX oraz wdrożenie produkcyjne na platformie Microsoft Azure z pipeline CI/CD w GitHub Actions.

2. Cel i zakres projektu

2.1. Cel biznesowy

Aplikacja ma wspierać osoby aktywnie trenujące w:

- ewidencji wykonanych treningów i poszczególnych ćwiczeń,
- monitorowaniu spożywanych posiłków i makroskładników,
- szybkim wyszukiwaniu produktów spożywczych w zewnętrznej bazie FatSecret,
- prezentacji dziennych podsumowań kalorycznych i makro w formie tabel oraz wykresów.

2.2. Zakres funkcjonalny (MVP)

W ramach projektu zrealizowano:

- moduł **Workouts** – CRUD treningów z zagnieżdżonymi wpisami ćwiczeń (siłowe i cardio),
- moduł **Meals** – CRUD posiłków i pozycji posiłku, filtrowanie oraz wykresy dziennej diety,
- moduł **Foods** – wyszukiwanie produktów w API FatSecret i dodawanie ich do posiłków,
- panel **Products** – zarządzanie cache’em produktów (tylko rola Admin),
- system **Identity** – rejestracja z potwierdzeniem e-mail, logowanie, role User i Admin.

3. Wymagania projektowe i ich realizacja

Poniżej zestawiono wymagania typowe dla projektu z przedmiotu programowania aplikacji webowych oraz sposób ich spełnienia w repozytorium `workout-tracker`.

Tabela 1: Mapowanie wymagań na elementy implementacji.

Wymaganie	Realizacja w projekcie
C# + interfejs ASP.NET	ASP.NET Core 8 MVC + Razor Pages (Identity UI)
Persystencja danych Odczyt/zapis + kolekcje	SQL Server + EF Core (Code First, migracje) CRUD, filtrowanie i wyszukiwanie (Workouts, Meals, Products)
Formularze ręczne Prezentacja danych	Widoki MVC z walidacją DataAnnotations Tabele, listy oraz wykresy Chart.js na stronie Meals
Uwierzytelnianie i role Zewnętrzne API + JSON Walidacja i obsługa błędów	ASP.NET Core Identity (User, Admin) FatSecret Platform API (OAuth 2.0, JSON) Atrybuty walidacyjne, serwisy domenowe, logowanie błędów API
Dokumentacja	Komentarze XML + strona DocFX w katalogu docfx/
Konteneryzacja Testy	Dockerfile + docker-compose.yml xUnit – 6 testów jednostkowych
Chmura obliczeniowa	Azure App Service + Azure SQL Database + GitHub Actions

4. Architektura systemu

4.1. Warstwy aplikacji

Projekt zastosował klasyczną architekturę MVC z wydzieloną warstwą serwisów:

- **Warstwa prezentacji** – kontrolery MVC (`WorkoutsController`, `MealsController`, `FoodsController`, `ProductsController`) oraz widoki Razor.
- **Warstwa logiki** – serwisy: `MealService`, `NutritionSummaryService`, `ProductCacheService`, `FatSecretClient`, `SmtEmailSender`.
- **Warstwa danych** – `ApplicationDbContext`, modele encji, migracje EF Core.
- **Uwierzytelnianie** – ASP.NET Core Identity z seedowaniem ról i opcjonalnego konta administratora.

4.2. Stos technologiczny

- **Backend:** .NET 8 LTS, ASP.NET Core MVC
- **Baza danych:** Microsoft SQL Server (LocalDB / Docker / Azure SQL)
- **ORM:** Entity Framework Core 8 (Code First)
- **Frontend:** Razor Views, Bootstrap 5, Chart.js, własny motyw CSS
- **API zewnętrzne:** FatSecret Platform API (OAuth 2.0 client credentials)

- **E-mail:** SMTP (np. Brevo) – potwierdzenie rejestracji konta
- **DevOps:** Docker, GitHub Actions, Azure App Service
- **Testy:** xUnit
- **Dokumentacja:** DocFX + komentarze XML

4.3. Przepływ danych przy wyszukiwaniu produktów

Przy dodawaniu produktu z FatSecret do posiłku wykonywane są następujące kroki:

1. Użytkownik wpisuje frazę wyszukiwania w module Foods.
2. Serwer pobiera token OAuth 2.0 i wywołuje REST API FatSecret.
3. Wyniki JSON są mapowane na modele domenowe i prezentowane w widoku.
4. Po wyborze produktu dane są cache'owane w tabeli `Products` przez `ProductCacheService`.
5. Pozycja posiłku (`MealItem`) jest tworzona z wartościami odżywczymi wybranej porcji.

5. Model danych

Model relacyjny obejmuje encje domenowe powiązane z użytkownikiem poprzez klucz obcy `UserId`. Główne tabele aplikacyjne:

- **Workout** – nazwa, data treningu, notatki; relacja 1:N do `ExerciseEntry`.
- **ExerciseEntry** – nazwa ćwiczenia, typ (*Strength* / *Cardio*), serie, powtórzenia, obciążenie, czas trwania, dystans, spalane kalorie.
- **Meal** – nazwa, data spożycia, notatki; relacja 1:N do `MealItem`.
- **MealItem** – nazwa, ilość, jednostka, kalorie, białko, węglowodany, tłuszcze; opcjonalne powiązanie z `Product`.
- **Product** – cache produktów z API (`ExternalId`, `Source`, makroskładniki, `CacheAt`).

Dodatkowo wykorzystywane są standardowe tabele ASP.NET Identity (`AspNetUsers`, `AspNetRoles` itd.). Schemat bazy jest wersjonowany migracjami EF Core uruchamianymi automatycznie przy starcie aplikacji.

6. Opis funkcjonalności

6.1. Moduł treningów (Workouts)

Zalogowany użytkownik może tworzyć, edytować, przeglądać i usuwać własne treningi. Każdy trening zawiera listę ćwiczeń z obsługą dwóch typów:

- **Siłowe** – liczba serii, powtórzenia (również per seria w formacie JSON), obciążenie w kg.
- **Cardio** – czas trwania w minutach, dystans w km, szacowane spalone kalorie.

Dostęp do danych innych użytkowników jest zablokowany na poziomie zapytań serwisowych (filtrowanie po `UserId`).

6.2. Moduł posiłków (Meals)

Moduł umożliwia pełny CRUD posiłków i ich składników. Na liście posiłków dostępne są filtry po nazwie i zakresie dat. Dla przefiltrowanych danych obliczane są dzienne sumy kalorii oraz makroskładników (białko, węglowodany, tłuszcze), prezentowane w:

- tabeli podsumowań dziennych,
- wykresie słupkowym kalorii (Chart.js),
- wykresie makroskładników w gramach.

6.3. Integracja z FatSecret

Klient `FatSecretClient` komunikuje się z API przy użyciu przepływu OAuth 2.0 *client credentials*. Token dostępu jest cache'owany w pamięci procesu, a odpowiedzi wyszukiwania i szczegółów produktu – z ograniczonym czasem ważności (odpowiednio 5 i 15 minut). Parsowanie odpowiedzi JSON realizowane jest na modelach `FatSecretFoodSearchResult` i `FatSecretFoodDetails`.

Ze względu na ograniczenia IP po stronie FatSecret, w panelu deweloperskim API należy dodać do whitelisty adresy **Outbound IP** z Azure App Service (oraz lokalny adres publiczny przy pracy w Dockerze).

6.4. Panel administratora (Products)

Użytkownik z rolą `Admin` ma dostęp do panelu zarządzania produktami zapisanymi lokalnie w cache:

- przegląd i filtrowanie po nazwie oraz źródle danych,
- edycja parametrów odżywczych produktu,
- bezpieczne usuwanie – powiązane `MealItem` są odpinane, lecz same posiłki pozostają.

Konto administratora może być automatycznie tworzone przy starcie aplikacji na podstawie konfiguracji `SeedAdmin:Email` i `SeedAdmin:Password`.

6.5. Rejestracja i logowanie

System Identity wymaga potwierdzenia adresu e-mail przed pierwszym logowaniem (`RequireConfirmedEmail = true`). Wysyłka wiadomości aktywacyjnej realizowana jest przez `SmtpEmailSender` (konfiguracja SMTP w zmiennych środowiskowych / App Settings).

7. Konteneryzacja i środowiska uruchomieniowe

7.1. Docker (środowisko deweloperskie)

Plik `docker-compose.yml` uruchamia dwa kontenery:

- **db** – SQL Server 2022 z healthcheckiem,
- **web** – aplikacja ASP.NET Core na porcie 8080.

Kontener aplikacji oczekuje na gotowość bazy danych, po czym automatycznie stosuje migracje EF Core. Sekrety (hasło SQL, klucze FatSecret, SMTP) przekazywane są przez plik `.env`.

7.2. Azure (środowisko produkcyjne)

Aplikacja została wdrożona na:

- **Azure App Service** (Linux, .NET 8) – hostowanie aplikacji,
- **Azure SQL Database** – baza produkcyjna (`workout-tracker-pwr-sql`),
- **GitHub Actions** – automatyczny build, test i deploy po pushu na gałąź `main`.

Konfiguracja produkcyjna obejmuje m.in.:

- `ConnectionStrings__DefaultConnection`
- `FatSecret__ClientId`, `FatSecret__ClientSecret`
- `Email__SmtpHost`, `Email__SmtpUsername`, `Email__SmtpPassword`, `Email__FromAddress`
- `SeedAdmin__Email`, `SeedAdmin__Password`
- `ASPNETCORE_ENVIRONMENT=Production`

Pipeline CI/CD (plik `.github/workflows/ci-cd.yml`) wykonuje kolejno: restore, build, test, publish oraz deploy do App Service z wykorzystaniem publish profile przechowywanego w GitHub Secrets.

8. Testy

W projekcie `WorkoutTracker.Tests` zaimplementowano 6 testów jednostkowych w frameworku xUnit:

- **FatSecretClientTests** (2 testy) – poprawność parsowania odpowiedzi JSON wyszukiwania i szczegółów produktu (z mockowanym `HttpClient`).
- **NutritionSummaryServiceTests** (4 testy) – agregacja dziennych sum kalorycznych i makro, obsługa pustych danych oraz wartości null.

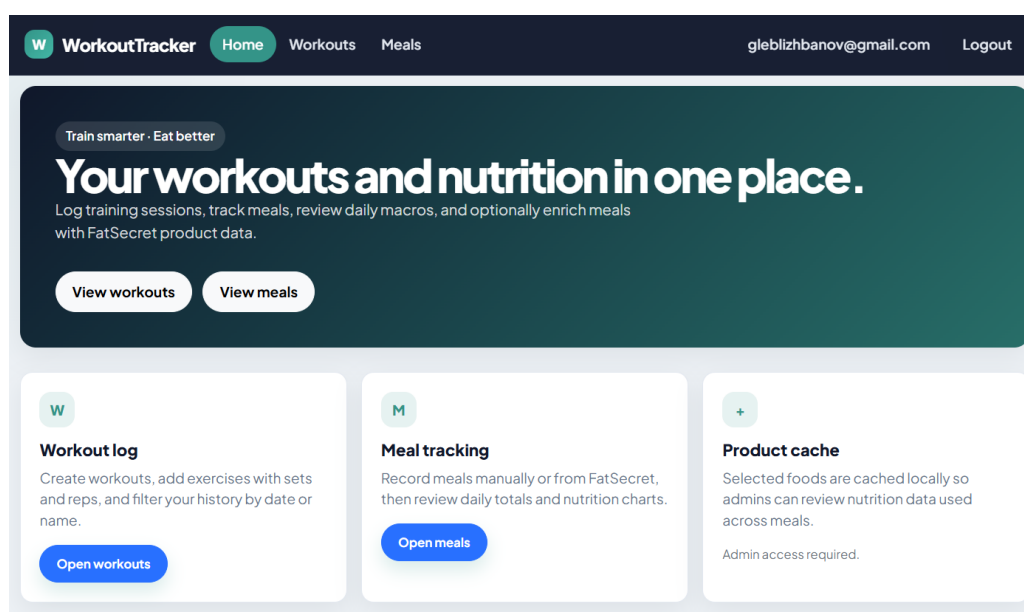
Testy uruchamiane są lokalnie poleceniem `dotnet test` oraz automatycznie w pipeline CI/CD przed każdym wdrożeniem na Azure.

9. Dokumentacja techniczna

Dokumentacja API została wygenerowana narzędziem DocFX na podstawie komentarzy XML osadzonych w kodzie źródłowym. Struktura dokumentacji znajduje się w katalogu `docfx/` i obejmuje:

- opis architektury i głównych modułów (`articles/introduction.md`),
- automatycznie wygenerowaną referencję klas, interfejsów i metod publicznych,
- konfigurację filtrującą migracje EF Core z dokumentacji API.

10. Zrzuty ekranu aplikacji



Rysunek 1: Strona główna aplikacji Workout Tracker po zalogowaniu.

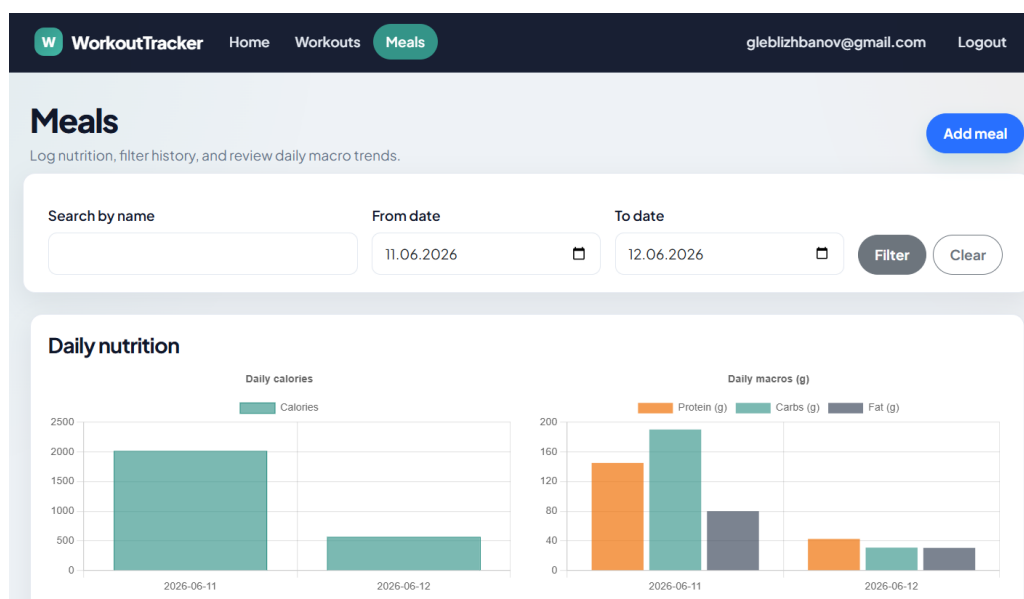
The screenshot shows the 'Workouts' page of the WorkoutTracker application. The header includes the logo 'W WorkoutTracker', navigation links 'Home', 'Workouts' (active), and 'Meals', along with the user email 'gleblizhbanov@gmail.com' and a 'Logout' button. The main heading is 'Workouts' with a subtitle 'Track sessions and review your training history.' and an 'Add workout' button. Below this is a filtering section with 'Search by name' (text input), 'From date' (calendar icon, placeholder 'dd.mm.rrrr'), 'To date' (calendar icon, placeholder 'dd.mm.rrrr'), a 'Filter' button, and a 'Clear' button. The main content is a table with columns 'NAME', 'DATE', and 'NOTES'. It lists two workouts: 'silownia' on '2026-06-12' with notes 'dzień nóg', and 'bieganie' on '2026-06-11' with notes 'park'. Each row has links for 'Details', 'Edit', and 'Delete'. The footer shows '© 2026 WorkoutTracker' and a 'Privacy' link.

NAME	DATE	NOTES	
silownia	2026-06-12	dzień nóg	Details Edit Delete
bieganie	2026-06-11	park	Details Edit Delete

Rysunek 2: Lista treningów użytkownika z możliwością filtrowania.

The screenshot shows the 'Add Workout' form in the WorkoutTracker application. The header is identical to the previous screenshot. The form has a title 'Bench press' in a text input field. Below it is the 'Exercise type' section with two radio buttons: 'Strength / gym' (selected) and 'Cardio / endurance'. The 'Number of sets' is a text input with the value '3'. The 'Repetitions per set' section has three text inputs for 'Set 1' (10), 'Set 2' (9), and 'Set 3' (9). The 'Weight (kg)' section has a text input with '80' and a unit selector 'kg'. Below this, it shows 'Estimated calories burned: ~43 kcal'. At the bottom is a 'Notes' section with a large text area.

Rysunek 3: Formularz dodawania ćwiczenia.



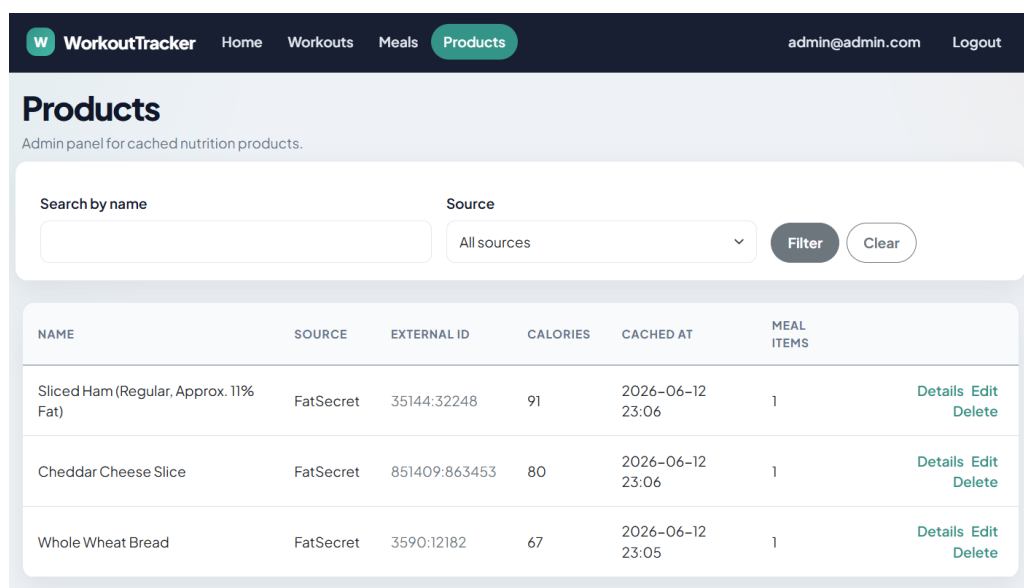
Rysunek 4: Moduł posiłków z tabelą podsumowań i wykresami dziennej diety.

FatSecret search
Meal: breakfast

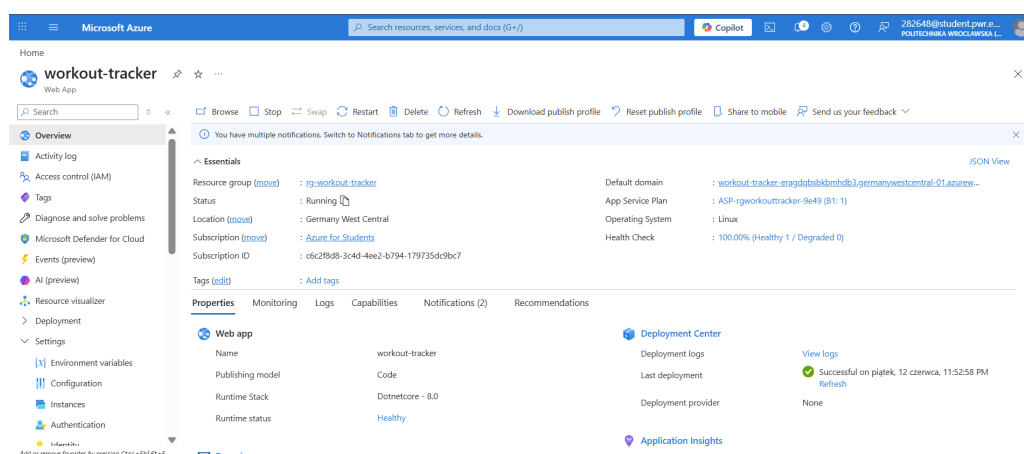
cheddar cheese Search Back

Name	Description	
Cheddar Cheese	Per 100g - Calories: 403kcal Fat: 33.14g Carbs: 1.28g Protein: 24.90g	View
Sharp Cheddar Cheese	Per 1 oz - Calories: 110kcal Fat: 9.00g Carbs: 0.00g Protein: 7.00g	View
Shredded Cheddar Cheese	Per 1/4 cup - Calories: 28kcal Fat: 2.25g Carbs: 2.50g Protein: 1.75g	View
Deli Style Sliced Natural Cheddar Cheese	Per 1 slice - Calories: 80kcal Fat: 7.00g Carbs: 0.00g Protein: 5.00g	View
Medium Cheddar Cheese	Per 1 oz - Calories: 110kcal Fat: 9.00g Carbs: 1.00g Protein: 6.00g	View
Finely Shredded Mild Cheddar Cheese	Per 1/4 cup - Calories: 110kcal Fat: 9.00g Carbs: 1.00g Protein: 6.00g	View
Extra Sharp Cheddar Cheese	Per 1 oz - Calories: 120kcal Fat: 10.00g Carbs: 1.00g Protein: 6.00g	View
Medium Cheddar Cheese	Per 1 oz - Calories: 110kcal Fat: 9.00g Carbs: 1.00g Protein: 6.00g	View
2% Milk Extra Sharp Cheddar Cheese	Per 1 oz - Calories: 90kcal Fat: 6.00g Carbs: 2.00g Protein: 7.00g	View

Rysunek 5: Wyszukiwanie produktów spożywczych w API FatSecret.



Rysunek 6: Panel administratora – zarządzanie cache’em produktów.

Rysunek 7: Wdrożona aplikacja na Azure App Service (<https://workout-tracker-eragdqbsbkbmhd3.azurewebsites.net>).

11. Wnioski

Na podstawie realizacji projektu *Workout Tracker* sformułowano następujące wnioski:

- **Kompletność funkcjonalna:** Aplikacja spełnia założenia projektu grupowego – umożliwia rejestrowanie treningów i posiłków, integruje się z zewnętrznym API, zapewnia role użytkowników oraz prezentuje dane w formie tabel i wykresów.
- **Architektura warstwowa:** Wydzielenie serwisów domenowych (*MealService*, *NutritionSummaryService*, *ProductCacheService*) uprościło kontrolery i umożliwiło testowanie logiki obliczeniowej niezależnie od bazy danych i HTTP.
- **Integracja zewnętrzna:** Połączenie z FatSecret wymagało obsługi OAuth 2.0, parsowania JSON oraz konfiguracji whitelisty IP – szczególnie po wdrożeniu na Azure, gdzie ruch wychodzący pochodzi z adresów Outbound IP App Service, a nie z przeglądarki użytkownika.

- **Bezpieczeństwo:** Zastosowanie ASP.NET Core Identity z potwierdzeniem e-mail, autoryzacją opartą na rolach oraz izolacją danych po `UserId` zapewnia podstawową ochronę danych osobowych użytkowników.
- **DevOps:** Konteneryzacja Docker ułatwiła lokalne uruchomienie całego stosu (aplikacja + SQL Server), natomiast pipeline GitHub Actions i Azure App Service umożliwiły powtarzalne wdrożenia produkcyjne bez ręcznej konfiguracji serwera.
- **Dalszy rozwój:** Możliwymi kierunkami rozbudowy są m.in. eksport danych treningowych, integracja z urządzeniami wearables, rozbudowa panelu administracyjnego o zarządzanie użytkownikami oraz testy integracyjne z bazą danych w środowisku CI.

Adres wdrożonej aplikacji: <https://workout-tracker-eragdqbbsbkbmhd3.azurewebsites.net>